



PROJECT

MACHINE LEARNING

Topic: Phishing Website Detection

Decoding Data: Learn AI, ML, DS & Analytics

Session: 2025–2029

B. Tech – Computer Science and Engineering (AI-ML)

Semester: 2nd

Section: H

Introduction

Phishing websites are fake or malicious websites created by cybercriminals to steal sensitive information such as usernames, passwords, banking details, OTPs, and personal data from users. These websites imitate trusted platforms like banks, shopping websites, social media platforms, and email services.

With the rapid increase in internet usage and online transactions, phishing attacks have become one of the most common cyber threats worldwide. Traditional security systems are often unable to detect newly created phishing websites quickly. Therefore, Machine Learning (ML) techniques are increasingly used to automatically identify and classify phishing websites.

In this project, a Machine Learning model is developed to detect phishing websites using URL-based features and text processing techniques. The project uses Natural Language Processing (NLP), tokenization, stemming, Count Vectorization, and classification algorithms such as Logistic Regression and Multinomial Naive Bayes.

The developed system can classify a website URL as either:

- Good (Legitimate Website)
- Bad (Phishing Website)

This project demonstrates how Artificial Intelligence and Data Science techniques can improve cybersecurity systems.

Objectives

The main objectives of this project are:

1. To study phishing attacks and their impact on cybersecurity.
2. To collect and preprocess website URL datasets.
3. To apply Natural Language Processing (NLP) techniques on URLs.
4. To extract useful features using Count Vectorizer.
5. To train Machine Learning models for phishing website detection.

6. To compare prediction accuracy of different ML algorithms.
7. To classify websites as legitimate or phishing.
8. To improve internet security using AI and Machine Learning.

Dataset Description

The dataset used in this project contains website URLs along with labels indicating whether the URL is safe or phishing.

Dataset Information

- Total Records: 549,346 URLs
- Features: 2 Columns
- Columns:
 - URL
 - Label

The labels are:

- good → Legitimate website
- bad → Phishing website

Dataset Shape

The dataset contains:

- Rows: 549,346
- Columns: 2

```
[3]: df.head()
```

	URL	Label
0	nobell.it/70ffb52d079109dca5664cce6f317373782/...	bad
1	www.dghjdgdg.com/paypal.co.uk/cycgi-bin/webscr...	bad
2	serviciosbys.com/paypal.cgi.bin.get-into.herf....	bad
3	mail.printakid.com/www.online.americanexpress....	bad
4	thewhiskeydregs.com/wp-content/themes/widescre...	bad

```
[4]: df.shape
```

```
[4]: (549346, 2)
```

```
[5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 549346 entries, 0 to 549345
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    URL      549346 non-null    object
1    Label    549346 non-null    object
dtypes: object(2)
memory usage: 8.4+ MB
```

Figure 1: Dataset Shape and Information

Missing Values

The dataset does not contain any missing values.

```
[6]:
```

```
df.isnull().sum()
```

```
[6]:
```

```
URL      0
Label    0
dtype: int64
```

Figure 2: Null Value Checking

Dataset Distribution

The dataset contains both legitimate and phishing URLs.

- Good URLs: 392,924
- Bad URLs: 156,422

This balanced dataset helps improve model performance.

Methodology

The project follows several steps for phishing website detection.

1. Importing Libraries

Python libraries used in the project include:

- Pandas
- NumPy
- Matplotlib
- NLTK
- Scikit-learn
- WordCloud

[1]:

```
#import basic libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

[2]:

```
df = pd.read_csv(r'C:\Users\swapn\Downloads\dataset.zip')
```

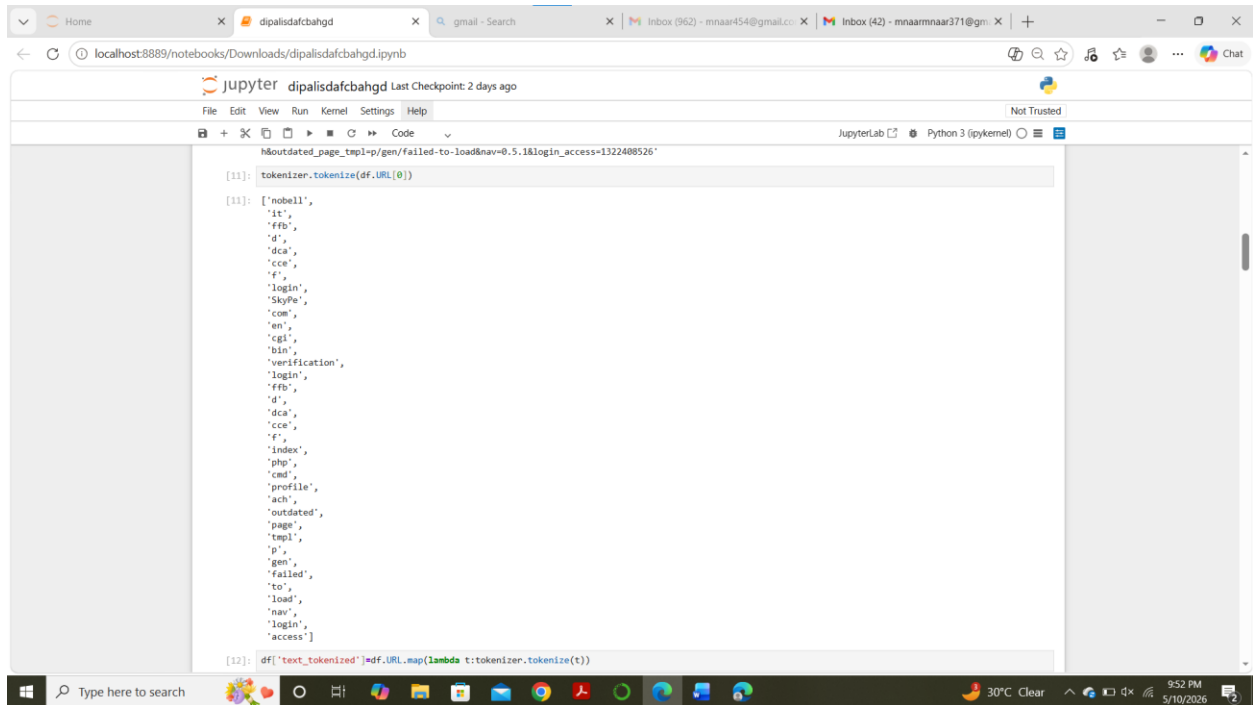
Figure 3: Importing Libraries and Loading Dataset

2. Data Preprocessing

The URLs were cleaned and processed before model training.

Tokenization

The URLs were split into meaningful tokens using Regular Expression Tokenizer.



```
tokenizer.tokenize(df.URL[0])

[11]: ['nobell',
      'it',
      'ffb',
      'd',
      'dca',
      'cce',
      'f',
      'login',
      'SkyPe',
      'com',
      'en',
      'cgi',
      'bin',
      'verification',
      'login',
      'ffb',
      'd',
      'dca',
      'cce',
      'f',
      'index',
      'php',
      'cmd',
      'profile',
      'ach',
      'outdated',
      'page',
      'templ',
      'p',
      'gem',
      'failed',
      'to',
      'load',
      'nav',
      'login',
      'access']

[12]: df['text_tokenized'] = df.URL.map(lambda t: tokenizer.tokenize(t))
```

Figure 4: URL Tokenization

3. Stemming

Words were reduced to their root form using Snowball Stemmer.

```
[ ]:
[14]: from nltk.stem.snowball import SnowballStemmer
[15]: stemmer = SnowballStemmer('english')
[16]: df['text_stemmed'] = df['text_tokenized'].map(lambda l: [stemmer.stem(word) for word in l])
[17]: df.head()
```

	URL	Label	text_tokenized	text_stemmed
0	nobell.it/70ffb52d079109dca5664cce6f31737382/...	bad	[nobell, it, ffb, d, dca, cce, f, login, SkyPe...	[nobel, it, ffb, d, dca, cce, f, login, skype...
1	www.dghjdgf.com/paypal.co.uk/cycgi-bin/webscr...	bad	[www, dghjdgf, com, paypal, co, uk, cycgi, bin...	[www, dghjdgf, com, paypal, co, uk, cycgi, bin...
2	serviciosbys.com/paypal.cgi.bin.get-into.herf...	bad	[serviciosbys, com, paypal, cgi, bin, get, int...	[serviciosbi, com, paypal, cgi, bin, get, into...
3	mail.printakid.com/www.online.americanexp...	bad	[mail, printakid, com, www, online, americanex...	[mail, printakid, com, www, onlin, americanexp...
4	thewhiskeydregs.com/wp-content/themes/widescr...	bad	[thewhiskeydregs, com, wp, content, themes, wi...	[thewhiskeydreg, com, wp, content, theme, wide...

Figure 5: Stemming Process

Word clouds were generated to visualize the most frequent terms used in good and bad URLs.

[illegible]

```
[28]: #generate word cloud
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(all_text)

#display the word cloud
plt.figure(figsize=(10,5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()
```

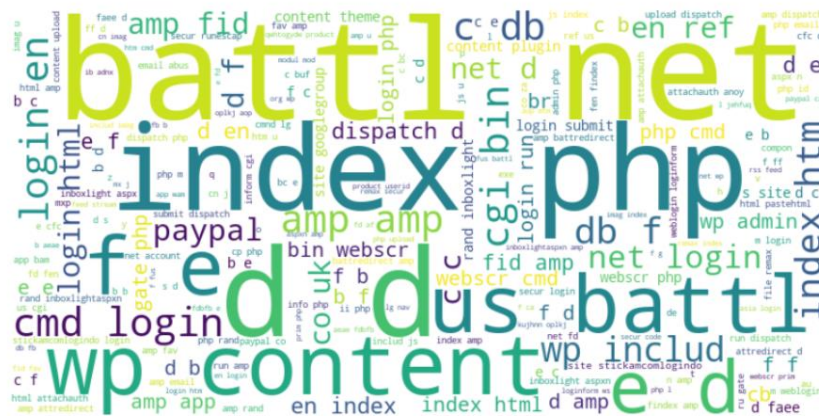


Figure 7: Word Cloud for Phishing Websites

5. Feature Extraction

- Converts text-based URLs into numerical data.
- Uses CountVectorizer to extract important words/tokens.
- Creates feature vectors from URL text.
- Helps machine learning models understand URL patterns.

It transforms URLs into machine-readable numerical features for phishing detection.

```
[30]: from sklearn.feature_extraction.text import CountVectorizer

[31]: cv = CountVectorizer()

[32]: features = cv.fit_transform(df.text)

[33]: features[:5].toarray()

[33]: array([[0, 0, 0, ..., 0, 0, 0],
           [0, 0, 0, ..., 0, 0, 0],
           [0, 0, 0, ..., 0, 0, 0],
           [0, 0, 0, ..., 0, 0, 0],
           [0, 0, 0, ..., 0, 0, 0]], shape=(5, 350837))
```

Figure 8: Count Vectorizer Features

6. Train-Test Split

The dataset was divided into:

- Training Data: 80%
- Testing Data: 20%

This helps evaluate model performance effectively.

```
[34]: from sklearn.model_selection import train_test_split

[35]: x_train, x_test, y_train, y_test = train_test_split(features, df.Label, test_size=0.2)
```

Figure 9: Train-Test Split

7. Model Training

Two Machine Learning models were used:

Logistic Regression

- Used for binary classification.
- Trained using extracted URL features.
- Predicts whether a website is phishing or legitimate.
- Provides high accuracy for text classification tasks.

Multinomial Naive Bayes

- Based on probability classification.
- Works efficiently with text data.
- Trained on Count Vectorizer features.
- Useful for detecting phishing patterns in URLs.

Model Training

```
[36]: from sklearn.linear_model import LogisticRegression
```

```
[37]: l_model = LogisticRegression()
```

```
[38]: l_model.fit(x_train,y_train)
```

C:\Users\swapn\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
    n_iter_i = _check_optimize_result(
LogisticRegression()
```

[38]: In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

LogisticRegression
?Documentation for LogisticRegressionFitted
► Parameters

Figure 10: Logistic Regression Training

```
[57]: mnb = MultinomialNB()
[58]: mnb.fit(x_train, y_train)
[58]: MultinomialNB()
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.
MultinomialNB
?Documentation for MultinomialNBifitted
► Parameters
[59]: mnb.score(x_test, y_test)
[59]: 0.9587876581414398
```

Figure 11: Naive Bayes Training and Prediction

8. Prediction

- The trained machine learning model predicts whether a URL is phishing or legitimate.
- Input URLs are transformed using CountVectorizer before prediction.
- The model classifies URLs as “Bad” (phishing) or “Good” (safe).
- Prediction results are displayed for testing and evaluation.

```
[69]: l_model.predict(predict_good)
[69]: array(['bad', 'bad', 'bad', 'good'], dtype=object)
[70]: l_model.predict(predict_bad)
[70]: array(['good', 'good', 'good', 'good'], dtype=object)
[ ]:
```

Figure 12: Prediction Results of Phishing and Legitimate URLs

Results

The trained models successfully classified phishing and legitimate URLs.

Logistic Regression Accuracy

- Training Accuracy: 97.92%
- Testing Accuracy: 96.54%

Classification Report

The model achieved high precision, recall, and F1-score.

- Precision: 97%
- Recall: 97%
- F1-Score: 97%

The confusion matrix and classification report indicate strong performance.

The model was also tested on sample URLs and produced accurate predictions.

Sample Predictions

Phishing URLs

- yenik.com.tr/wp-admin/js/login.alibaba.com/login.jsp.php
- fazan-pacir.rs/temp/libraries/ipad
- www.tubemoviez.exe

Predicted Output: bad

Legitimate URLs

- www.youtube.com
- youtube.com/watch
- www.retailhellunderground.com

Predicted Output: good

```
[42]: print('\nCLASSIFICATION REPORT\n')
print(classification_report(l_model.predict(x_test),y_test,
                             target_names=['Bad','Good']))
```

```
CLASSIFICATION REPORT
```

	precision	recall	f1-score	support
Bad	0.91	0.97	0.94	29293
Good	0.99	0.96	0.98	80577
accuracy			0.97	109870
macro avg	0.95	0.97	0.96	109870
weighted avg	0.97	0.97	0.97	109870

Figure 13: Classification Report and Model Accuracy Results

Limitations

Although the project achieved good accuracy, there are still some limitations:

1. The model mainly depends on URL-based features.
2. Newly generated phishing websites may bypass detection.
3. The dataset may contain outdated URLs.
4. The model does not analyze webpage content deeply.
5. Real-time detection may require additional optimization.
6. False positives and false negatives may still occur.

Conclusion

This project successfully developed a Machine Learning-based phishing website detection system using NLP and classification techniques.

The system effectively distinguishes phishing websites from legitimate websites with high accuracy. Techniques such as tokenization, stemming, Count Vectorization, Logistic Regression, and Naive Bayes significantly improved prediction performance.

The project demonstrates the practical use of Artificial Intelligence, Machine Learning, and Data Science in the field of cybersecurity.

In the future, this system can be improved by:

- Using Deep Learning models
- Analyzing webpage content and HTML structure
- Implementing real-time phishing detection
- Integrating browser extensions for live protection

Overall, the project provides an efficient and intelligent approach for detecting phishing websites and enhancing online security.

Tools and Technologies Used

- Python
- Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- NLTK (Natural Language Toolkit)
- Scikit-learn
- CountVectorizer
- Logistic Regression
- Multinomial Naive Bayes
- WordCloud
- Machine Learning
- Natural Language Processing (NLP)